

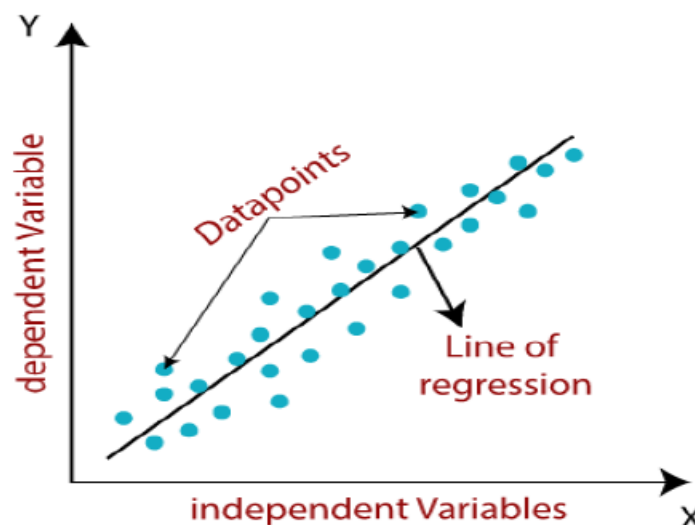
UNIT-2: SUPERVISED LEARNING

Regression:

- **Regression** is a statistical approach used in machine learning to analyze the relationship between a **dependent variable** (also known as the target variable) and one or more **independent variables** (predictor variables).
- The primary objective of regression is to determine the most suitable function that characterizes the connection between these variables allowing for predictions or conclusions.

Linear regression:

- One of the simplest regression models is **linear regression** which aims to find the best-fitting linear relationship between the input features and the target variable.
- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- The linear regression model provides a sloped straight line representing the relationship between the variables. Consider the below image:



Mathematically, we can represent a linear regression as:

$$Y = aX + b$$

Here, Y = dependent variables (target variables),

X = Independent variables (predictor variables),

a and b are the linear coefficients

- Linear regression is a statistical regression method which is used for predictive analysis.
- It is one of the very simple and easy algorithms which works on regression and shows the relationship between the continuous variables.
- It is used for solving the regression problem in machine learning.
- If there is only one input variable (x), then such linear regression is called **simple linear regression**. And if there is more than one input variable, then such linear regression is called **multiple linear regression**.

Applications of linear regression

- Analyzing trends and sales estimates
- Salary forecasting
- Real estate prediction
- Arriving at ETAs in traffic.

Non-linear regression

- Non-linear regression is a statistical method used to model relationships between independent and dependent variables when the relationship is not linear.
- Unlike linear regression, which fits data to a straight line, non-linear regression fits the data to a curve or any complex function.
- These models are incredibly versatile and are often used when the data exhibits curvature or follows specific patterns.

- If the underlying pattern in the data exhibits a curve whether it's exponential growth, decay, logarithmic or any other non-linear form, fitting a nonlinear regression model can provide a more accurate representation of the relationship.

Key Features:

1. **Flexible models:** Allows for fitting data that doesn't align to a straight line (e.g., exponential, logarithmic, polynomial, sigmoidal functions).
2. **Applications:** Commonly used in biology (e.g., enzyme kinetics), economics (e.g., growth modeling), physics (e.g., decay rates) and machine learning.
3. **Estimation Process:** Employs iterative methods like gradient descent or the Levenberg-Marquardt algorithm to find the best-fit parameters.
4. **Example Functions:**
 - Quadratic: $y = ax^2 + bx + c$
 - Exponential: $y = ae^{bx}$
 - Logistic: $y = L / 1 + e^{-k(x - x_0)}$

Implementation Non-Linear Regression using Python:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# Read the CSV file
df = pd.read_csv('/content/gdp.csv')
# Display the first few rows of the dataframe
print(df.head())
```

Model Evaluation Method

- Model evaluation methods are techniques used to assess the performance of a statistical, machine learning or predictive model.
- These methods help determine how well the model predicts outcomes, generalizes to new data and aligns with the problem's requirements.
- Some commonly used model evaluation methods are:

1. Train-Test Split

- The dataset is divided into two parts: one for training the model and another for testing it.
- The training set is used to train the model, while the test set serves as an independent dataset for evaluating the trained model's performance.
- The allocation of data to the training set and the test set is typically based on a predefined ratio such as 70:30, 80:20 or 90:10.
- The training set contains a larger portion of the data, allowing the model to learn patterns and relationships from a substantial amount of information.
- Helps identify how well the model generalizes to unseen data.

2. Cross-Validation

- Divides the data into "folds," trains the model on some folds, and tests it on the remaining fold(s).
- Methods include:
 - ***k-Fold Cross-Validation:*** Data is split into k parts, and the process is repeated k times.
 - ***Leave-One-Out Cross-Validation (LOOCV):*** Uses one data point for testing and the rest for training, repeated for all points.
- Reduces variability and gives a more reliable performance estimate.

3. Performance Metrics

- Different metrics are used depending on the problem type:
 - **For Regression:** Mean Squared Error (MSE), Mean Absolute Error (MAE), R-squared.
 - **For Classification:** Accuracy, Precision, Recall, F1-score, ROC-AUC.
 - **For Clustering:** Silhouette Score, Davies-Bouldin Index.

4. Confusion Matrix (for Classification)

- A confusion matrix is a powerful tool for model evaluation of classification.
- It provides a tabular representation of the predicted and actual class labels enabling a detailed analysis of model performance by showing:
 - True Positives (TP)
 - False Positives (FP)
 - True Negatives (TN)
 - False Negatives (FN)

- Useful for computing metrics like precision and recall.

		True Class	
		Positive	Negative
Predicted Class	Positive	True Positive (TP)	False Positive (FP)
	Negative	False Negative (FN)	True Negative (TN)

Fig: General structure of a confusion matrix

- A confusion matrix is typically organized into four quadrants: true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN).
- Each quadrant represents a specific outcome of the classification process.
- TP indicates the number of correctly predicted positive instances, TN represents the correctly predicted negative instances, FP denotes the instances incorrectly predicted as positive and FN indicates the instances incorrectly predicted as negative.

From the confusion matrix, various model evaluation metrics can be derived. Some commonly used metrics include:

▪ **Accuracy:**

- The overall accuracy of the model, calculated as $(TP + TN) / (TP + TN + FP + FN)$.
- It provides an indication of the model's overall predictive performance.

▪ **Precision:**

- It is also known as positive predictive value, it measures the proportion of correctly predicted positive instances out of all instances predicted as positive.
- Precision is calculated as $TP / (TP + FP)$ and is useful when the focus is on minimizing false positives.

▪ **Recall:**

- It is also known as sensitivity or true positive rate, it measures the proportion of correctly predicted positive instances out of all actual positive instances.
- Recall is calculated as $TP / (TP + FN)$ and is useful when the focus is on minimizing false negatives.

▪ **F1 Score:**

- The harmonic mean of precision and recall providing a balanced measure of the models performance.
- It is calculated as $2 * (Precision * Recall) / (Precision + Recall)$.

- The confusion matrix offers insights into the specific types of errors made by the model. It helps identify whether the model is more prone to false positives or false negatives, which can guide further model refinement or decision-making.
- **ROC-Curve:**
 - The receiver operating characteristic (ROC) curve is a widely used tool for evaluating the performance of binary classification models.
 - It provides a graphical representation of the trade-off between the true positive rate (sensitivity) and the false positive rate (1-specificity) at various classification thresholds.
 - The ROC curve is created by plotting the true positive rate (TPR) on the y-axis against the false positive rate (FPR) on the x-axis as the classification threshold is varied.
 - Each point on the ROC curve corresponds to a specific threshold setting, indicating the model's performance at that threshold.
 - The ideal model would have a ROC curve that hugs the top-left corner of the plot, representing high TPR and low FPR across all threshold values.

5. Bias-Variance Tradeoff

- Evaluates a model for its balance between under-fitting (bias) and overfitting (variance).
- Helps in selecting the right model complexity.

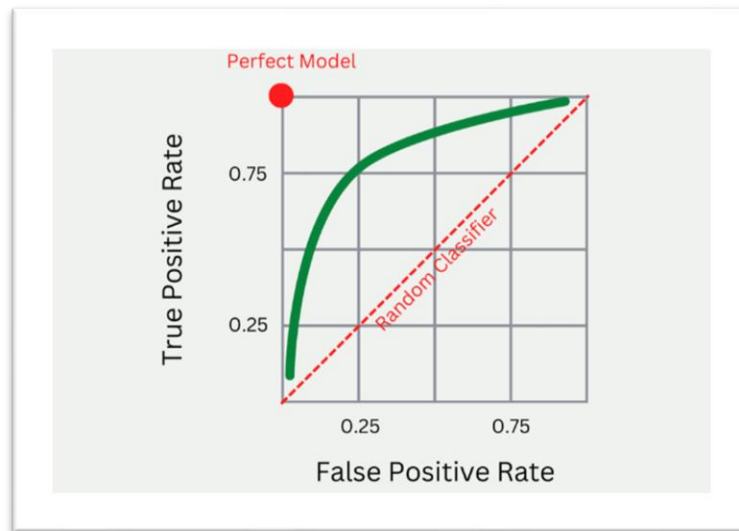
6. Holdout Method

- Similar to train-test split but often includes a validation set to tune hyper-parameters before final testing.

Structure of a ROC-Curve

- One of the key advantages of the ROC curve is its ability to provide a comprehensive assessment of a model's performance across various threshold settings.
- It allows practitioners to visualize the trade-off between sensitivity and specificity and choose an appropriate threshold that balances the two based on the problem context and requirements.

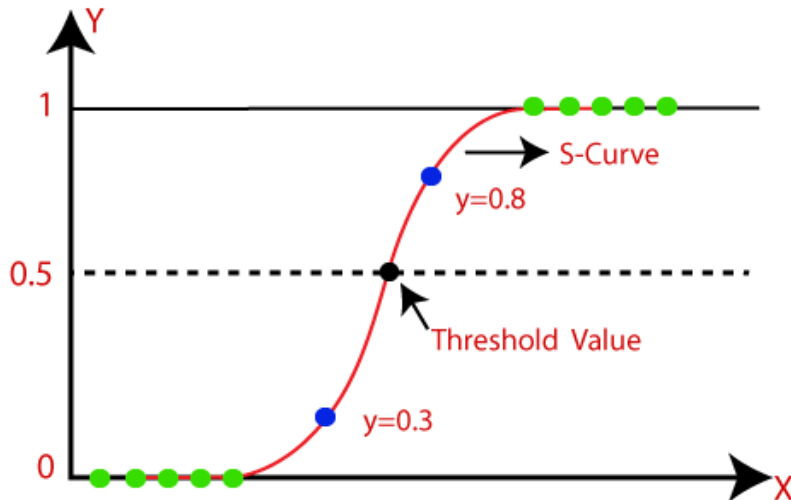
- The area under the ROC curve (AUC-ROC) is a commonly used summary metric derived from the ROC curve.
- It represents the overall performance of the model, regardless of the threshold setting.
- An AUC-ROC score of 0.5 indicates a random classifier, while a score of 1.0 represents a perfect classifier.
- The closer the AUC-ROC score is to 1.0, the better the model's ability to distinguish between positive and negative instances.
- The ROC curve and AUC-ROC provide several benefits in model evaluation.
- They are robust to class imbalance and are not affected by the specific choice of threshold.
- The curve offers a visual representation of the model's discrimination ability and enables the comparison of multiple models.
- The AUC-ROC score provides a single-value summary metric that simplifies model comparison and selection.
- However, it's important to note that the ROC curve and AUC-ROC are most suitable for binary classification tasks.
- They may not be directly applicable to multi-class problems without appropriate modifications, such as the one-vs-rest or one-vs-one strategies.
- Additionally, the interpretation of the ROC curve and AUC-ROC should be considered in conjunction with the specific requirements and costs associated with false positives and false negatives.



Classification

Logistic regression:

- Logistic regression is one of the most popular machine learning algorithm that comes under supervised learning techniques.
- It can be used for classification as well as for regression problems but mainly used for classification problems.
- Logistic regression is used to predict the categorical dependent variable with the help of independent variables.
- The output of logistic regression problem can be only between the 0 and 1.
- Logistic regression can be used where the probabilities between two classes is required. Such as whether it will rain today or not, either 0 or 1, true or false etc.
- Logistic regression is based on the concept of Maximum Likelihood estimation. According to this estimation, the observed data should be most probable.
- In logistic regression, we pass the weighted sum of inputs through an activation function that can map values in between 0 and 1. Such activation function is known as **sigmoid function** and the curve obtained is called as sigmoid curve or S-curve. Consider the below image:



The equation for logistic regression is:

$$\log \left[\frac{y}{1-y} \right] = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

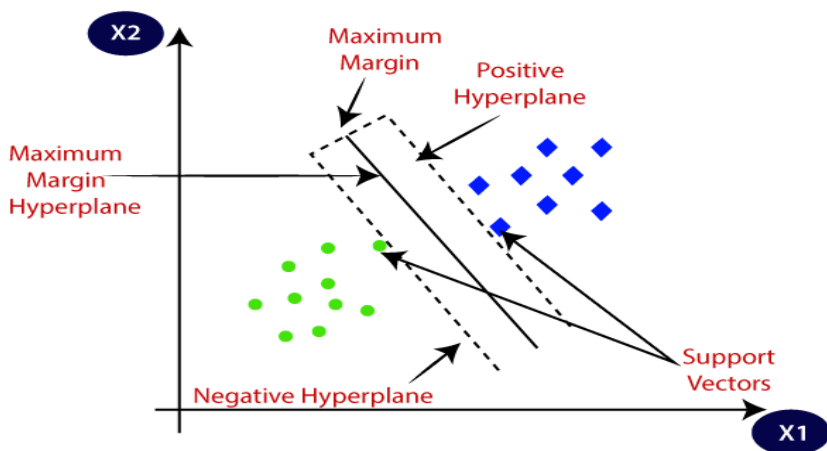
- **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
- **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep".
- **Ordinal:** In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

Difference between Linear Regression and Logistic Regression:

Linear Regression	Logistic Regression
Linear regression is used to predict the continuous dependent variable using a given set of independent variables.	Logistic Regression is used to predict the categorical dependent variable using a given set of independent variables.
Linear Regression is used for solving Regression problem.	Logistic regression is used for solving Classification problems.
In Linear regression, we predict the value of continuous variables.	In logistic Regression, we predict the values of categorical variables.
In linear regression, we find the best fit line, by which we can easily predict the output.	In Logistic Regression, we find the S-curve by which we can classify the samples.
Least square estimation method is used for estimation of accuracy.	Maximum likelihood estimation method is used for estimation of accuracy.
The output for Linear Regression must be a continuous value, such as price, age, etc.	The output of Logistic Regression must be a Categorical value such as 0 or 1, Yes or No, etc.
In Linear regression, it is required that relationship between dependent variable and independent variable must be linear.	In Logistic regression, it is not required to have the linear relationship between the dependent and independent variable.
In linear regression, there may be collinearity between the independent variables.	In logistic regression, there should not be collinearity between the independent variable.

Support Vector Machine (SVM):

- Support Vector Machine (SVM) is a powerful supervised learning algorithm used primarily for classification and regression tasks.
- The goal of SVM is to find the optimal hyperplane (best decision boundary) that separates data points into different classes in a high-dimensional space.
- SVM chooses the extreme points/vectors that help in creating the hyperplane.
- These extreme cases are called as support vectors and hence algorithm is termed as support vector machine.
- Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



- SVM algorithm can be used for **Face detection, image classification, text categorization, Bioinformatics (e.g., protein classification), Stock market predictions, etc.**
- SVM is robust and effective especially when the number of features is much larger than the number of samples.

Application of Support Vector Machine (SVM)

Face Detection: Classifying parts of an image as faces and non-faces.

Handwriting Recognition: Recognizing handwritten characters.

Image Classification: Enhancing performance in image classification tasks.

Bioinformatics: Used for protein classification and cancer classification.

Text and Hypertext Categorization: Classifying documents into different categories based on content.

Geo-spatial Data Applications: Utilized in various geo-spatial analysis tasks.

Security Applications: Employed in security systems for anomaly detection.

These applications demonstrate the versatility of SVM in handling various data types and tasks.

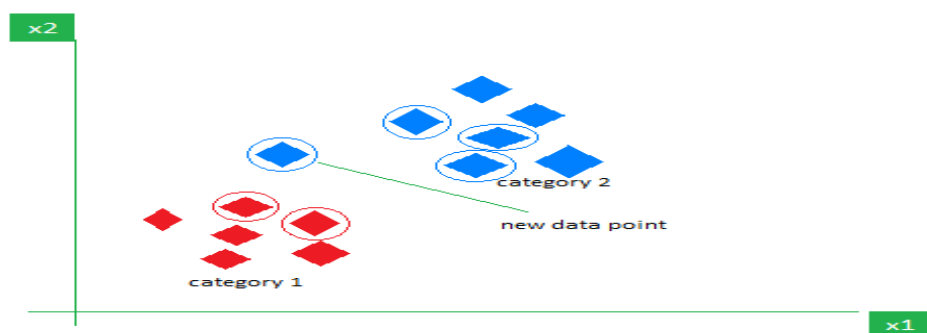
Python Implementation of Support Vector Machine

```
# importing libraries
import numpy as np
import matplotlib.pyplot as mtp
import pandas as pd
#importing datasets
data_set= pd.read_csv("C:/Users/Acer/Desktop/Weka/User_Data.csv")
#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
#feature Scaling
from sklearn.preprocessing import StandardScaler
```

```
st_x= StandardScaler()  
x_train= st_x.fit_transform(x_train)  
x_test= st_x.transform(x_test)
```

K-Nearest Neighbor (KNN):

- K-Nearest Neighbor (KNN) is a simple yet effective machine learning algorithm used for both classification and regression tasks.
- It's a non-parametric method, meaning it doesn't make any assumptions about the underlying data distribution.
- K-Nearest Neighbors is one of the simplest machine learning algorithms based on supervised learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- KNN predicts the output for a given input by looking at the “K” closest data points (neighbors) in the dataset. The distance between data points is measured using metrics like Euclidean distance, Manhattan distance or Minkowski distance.
- K-NN algorithm can be used for regression as well as for classification but mostly it is used for the classification problems.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- Example: Consider the following table of data points containing two features:



Applications of K-Nearest Neighbors (KNN)

Classification: KNN is widely used for classification tasks, where it predicts the class of a new data point based on the majority of its nearest neighbors.

Regression: KNN can also be applied to regression tasks, where it predicts the target variable by averaging the values of its nearest neighbors.

Recommendation Systems: KNN is used in recommendation systems to suggest items based on user behavior and preferences.

Image Classification: KNN is effective for image classification tasks, where it assigns labels to images based on their content.

Anomaly Detection: KNN can be utilized for anomaly detection by identifying data points that deviate significantly from the majority of the dataset.

KNN is a versatile algorithm that remains popular due to its simplicity and effectiveness in various real-world applications.

Numerical Example: The K-Nearest Neighbors (KNN) algorithm to classify a data point.

Let's consider we have a dataset with the following points:

Point	Feature x1	Feature x2	Class
A	2.0	4.0	0
B	4.0	4.0	0

Point	Feature x1	Feature x2	Class
C	4.0	2.0	1
D	6.0	2.0	1

And we want to classify a new point $X = (5.0, 3.0)$ using KNN with $K = 3$.

Steps

1. **Calculate distances:** Use Euclidean distance formula:

$$\text{Distance} = \sqrt{(x_1 - x_1^{\text{new}})^2 + (x_2 - x_2^{\text{new}})^2}$$

Compute distances between X and the existing points:

- Distance to A = $\sqrt{(2.0 - 5.0)^2 + (4.0 - 3.0)^2} = 9 + 1 = 10 \approx 3.16$
- Distance to B = $\sqrt{(4.0 - 5.0)^2 + (4.0 - 3.0)^2} = 1 + 1 = 2 \approx 1.41$
- Distance to C = $\sqrt{(4.0 - 5.0)^2 + (2.0 - 3.0)^2} = 1 + 1 = 2 \approx 1.41$
- Distance to D = $\sqrt{(6.0 - 5.0)^2 + (2.0 - 3.0)^2} = 1 + 1 = 2 \approx 1.41$

2. **Find the 3 nearest neighbors:** Sort the distances:

- B(1.41), C(1.41), D(1.41)

The 3 nearest neighbors are B, C and D.

3. **Classify based on majority voting:** Among the 3 nearest neighbors:

- B belongs to class 0
- C belongs to class 1
- D belongs to class 1

The majority class is 1.

Conclusion: The new point $X = (5.0, 3.0)$ is classified as Class 1.

KNN Algorithm implementation using python

Importing necessary libraries

import numpy as np

import pandas as pd


```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Load the dataset (replace this with your actual dataset)
data_set = pd.read_csv("C:/Users/Acer/OneDrive/Desktop/NL/User_Data.csv")
# Extracting features (independent variables) and target (dependent variable)
X = data_set.iloc[:, [2, 3]].values # Independent variables
y = data_set.iloc[:, 4].values      # Dependent variable (target)
# Splitting the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
# Feature scaling (important for KNN to perform well)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Training the KNN model
k = 5 # Number of neighbors (can be tuned)
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train, y_train)
# Making predictions
y_pred = knn.predict(X_test)
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

```

```
print("\nClassification Report:")  
print(classification_report(y_test, y_pred))
```

How It Works:

- For **classification**: The class of the input is determined by a majority vote from its nearest neighbors. The input is assigned to the class most common among the K neighbors.
- For **regression**: The output value is calculated as the average (or weighted average) of the values of its K nearest neighbors.

Key Features:

- Simple to implement and intuitive.
- Sensitive to the choice of K (the number of neighbors). Choosing the right K value is essential for optimal performance.
- Computationally intensive for large datasets because it requires storing all the data and calculating distances for every query.

Applications:

- Handwritten digit recognition.
- Recommender systems.
- Medical diagnosis, such as classifying patients into risk categories.

Decision Tree Technique

- A decision tree is a flowchart, like tree structure where each internal node denotes the feature, branches denote the rules and the leaf nodes denote the result of the algorithm.
- It is a versatile supervised machine-learning algorithm, which is used for both classification and regression problems.

- It is one of the very powerful algorithms. And it is also used in Random Forest to train on different subsets of training data, which makes random forest one of the most powerful algorithms in machine learning.

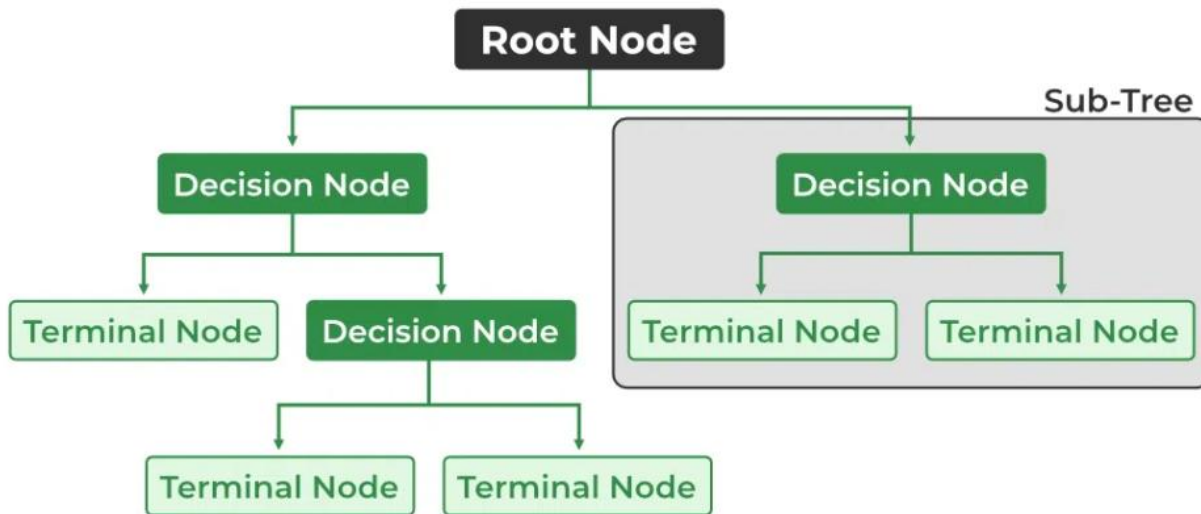


Fig: Decision Tree

Decision Tree Representation (Terminologies)

Some of the common terminologies used in Decision Trees are as follows:

- **Root Node:**

- The starting point of the tree, representing the initial decision or question.

- **Internal Nodes:**

- Intermediate points that represent decisions or tests based on attributes.

- **Branch/Sub-Tree:**

- A subsection of the decision tree starts at an internal node and ends at the leaf nodes.

- **Leaf Nodes:**

- Terminal points that represent final outcomes or classifications.

- **Splitting:**

- The process of splitting a node into two or more sub-nodes using a split criterion and a selected feature.
- **Parent Node:** The node that divides into one or more child nodes.
- **Child Node:** The nodes that emerge when a parent node is split.

Construction of Decision Tree:

- A tree can be “*learned*” by splitting the source set into subsets based on Attribute Selection Measures.
- Attribute selection measure (ASM) is a criterion used in decision tree algorithms to evaluate the usefulness of different attributes for splitting a dataset.
- The goal of ASM is to identify the attribute that will create the most homogeneous subsets of data after the split, thereby maximizing the information gain.
- This process is repeated on each derived subset in a recursive manner called *recursive partitioning*.
- The recursion is completed when the subset at a node all has the same value of the target variable, or when splitting no longer adds value to the predictions.
- The construction of a decision tree classifier does not require any domain knowledge or parameter setting and therefore is appropriate for exploratory knowledge discovery.
- Decision trees can handle high-dimensional data.

Appropriate problems for Decision Tree learning

→ Decision Tree learning is well-suited for specific types of problems. Here are some scenarios where it shines:

1. Attribute-Value Pair Representation:

- Instances are described by a fixed set of attributes and their values (e.g., temperature: hot, mild, cold).
- Decision Trees handle both discrete and continuous attributes effectively.

2. Discrete Output Values:

- Ideal for classification tasks where the target function has distinct categories (e.g., yes/no, spam/not spam).
- It can also handle multi-class problems.

3. **Disjunctive Descriptions:**

- Decision Trees naturally represent disjunctive expressions, making them suitable for problems requiring such descriptions.

4. **Handling Errors:**

- Robust to errors in training data, whether in classification labels or attribute values.

5. **Missing Attribute Values:**

- Can work even when some training examples have unknown attribute values.

6. **Noisy Data:**

- Effective in scenarios where data contains noise or inconsistencies.

Basic decision tree learning algorithm

The basic Decision Tree learning algorithm is a recursive process that builds a tree structure by splitting data based on attribute values.

Steps in the Algorithm

1. **Start with the Root Node:**

- Begin with the entire dataset and select the best attribute to split the data. The selection is based on criteria like information gain or Gini index.

2. **Split the Data:**

- Divide the dataset into subsets based on the chosen attribute's values.

3. **Create Child Nodes:**

- For each subset, create child nodes and repeat the process recursively.

4. **Stopping Criteria:**

- Stop splitting when:

- All instances in a node belong to the same class.
- There are no more attributes to split on.
- A predefined depth or size limit is reached.

5. **Assign Class Labels:**

- At leaf nodes, assign class labels based on the majority class in the subset.

Popular Algorithms

- **ID3:** Uses information gain to select attributes.
- **CART:** Uses Gini index or mean squared error for splitting.
- **C4.5:** An improvement over ID3, handling continuous attributes and missing values.

ID3 algorithm

The ID3 (Iterative Dichotomiser 3) algorithm is a popular method for constructing decision trees in machine learning.

Key Concepts

1. **Entropy:**

- Measures the impurity or randomness in the dataset. Lower entropy means more homogeneity.

2. **Information Gain:**

- The reduction in entropy achieved by splitting the dataset based on an attribute. The attribute with the highest information gain is selected for splitting.

Steps in the Algorithm

1. **Start with the Root Node:**

- Begin with the entire dataset and calculate the entropy for the target attribute.

2. **Select the Best Attribute:**

- For each attribute, calculate the information gain. Choose the attribute with the highest information gain to split the data.

3. **Split the Data:**

- Divide the dataset into subsets based on the values of the selected attribute.

4. Create Child Nodes:

- For each subset, repeat the process recursively to create child nodes.

5. Stopping Criteria:

- Stop when:
 - All instances in a node belong to the same class.
 - There are no more attributes to split on.
 - A predefined depth or size limit is reached.

6. Assign Class Labels:

- At leaf nodes, assign the class label based on the majority class in the subset.

Advantages

- Simple and easy to understand.
- Handles both categorical and numerical data.

Limitations

- Prone to overfitting if the tree becomes too complex.
- Does not handle continuous attributes or missing values as effectively as its successor,

C4.5.

Example: Simple example of the ID3 (Iterative Dichotomiser3) algorithm for creating a decision tree. This algorithm uses entropy and information gain to decide which attribute to split the data on at each step. Let's consider a small dataset of weather conditions and the decision to play tennis:

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Steps to Build the Decision Tree using ID3:

- 1. Calculate the Entropy of the Entire Dataset:** Entropy is a measure of the amount of uncertainty or randomness in the dataset. The formula for entropy is:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where p_i is the proportion of examples in class i .

- **H(S):** This represents the entropy of the system or dataset S. It quantifies uncertainty or disorder.

- **Σ (Summation symbol):** This indicates that we are summing up values across multiple classes i .
- **i :** Represents an index running from 1 to c , where c is the total number of classes in the dataset.
- **p_i :** The probability of an instance belonging to class i . This is calculated as the proportion of occurrences of class i in the dataset.
- **$\log_2(p_i)$:** The logarithm (base 2) of p_i , used to measure the information content of each class.
- **Negative sign (-):** Ensures that entropy is always a non-negative quantity as probabilities p_i are between 0 and 1, making their logarithms negative.

2. Calculate the Entropy for Each Attribute: For each attribute, we calculate the weighted average entropy of the subsets that result from splitting the dataset by that attribute.

3. Calculate Information Gain for Each Attribute: Information gain is the reduction in entropy achieved by partitioning the examples according to an attribute. It is calculated as:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where S_v is the subset of S for which attribute A has value v .

4. Select the Attribute with the Highest Information Gain: This attribute is used to split the dataset. The process is repeated recursively for each subset created by the split.

Example Calculation:

Step 1: Calculate Entropy of the Entire Dataset

Values (Outlook): Sunny, Overcast, Rain

$S = [9+, 5-]$

$\text{Entropy}(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.94$

Step 2: Calculate Entropy for Each Attribute

$S_{\text{Sunny}} \leftarrow [2+, 3-]$

$$\text{Entropy}(S_{\text{Sunny}}) = - (2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$$

$S_{\text{Overcast}} \leftarrow [4+, 0-]$

$$\text{Entropy}(S_{\text{Overcast}}) = - (4/4)\log_2(4/4) - (0/4)\log_2(0/4) = 0$$

$S_{\text{Rain}} \leftarrow [3+, 2-]$

$$\text{Entropy}(S_{\text{Rain}}) = - (3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971$$

Step 3: Calculate Information Gain for "Outlook":

Information Gain

$$\text{Gain}(S, \text{Outlook}) = \text{Entropy}(S) - \sum_{v \in \{\text{Sunny}, \text{Overcast}, \text{Rain}\}} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S, \text{Outlook}) = 0.94 - \left(\frac{5}{14} \cdot 0.971 \right) - \left(\frac{4}{14} \cdot 0 \right) - \left(\frac{5}{14} \cdot 0.971 \right)$$

$$\text{Gain}(S, \text{Outlook}) = 0.2464$$

Step 4: Repeat for Other Attributes

You would repeat this process for the other attributes ("Temperature", "Humidity", "Wind") and choose the attribute with the highest information gain to split the dataset. Similarly, we need to calculate Information Gain of remaining 3 attributes as follows:

Attribute: Temp

Values (Temp): Hot, Mild, Cool

$S = [9+, 5-]$

$$\text{Entropy}(S) = - 9/14\log_2(9/14) - 5/14\log_2(5/14) = 0.94$$

$S_{\text{Hot}} \leftarrow [2+, 2-]$

$$\text{Entropy}(S_{\text{Hot}}) = - 2/4\log_2(2/4) - 2/4\log_2(2/4) = 1.0$$

$S_{\text{Mild}} \leftarrow [4+, 2-]$

$$\text{Entropy}(S_{\text{Mild}}) = -4/6\log_2(4/6) - 2/6\log_2(2/6) = 0.9183$$

$$S_{\text{Cool}} \leftarrow [3+, 1-]$$

$$\text{Entropy}(S_{\text{Cool}}) = -3/4\log_2(3/4) - 1/4\log_2(1/4) = 0.8113$$

Information Gain

$$\text{Gain}(S, \text{Temp}) = \text{Entropy}(S) - \sum_{v \in \{\text{Hot}, \text{Mild}, \text{Cool}\}} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S, \text{Temp}) = 0.94 - \left(\frac{4}{14} \cdot 1.0 \right) - \left(\frac{6}{14} \cdot 0.9183 \right) - \left(\frac{4}{14} \cdot 0.8113 \right)$$

$$\text{Gain}(S, \text{Temp}) = 0.029$$

Attribute: Humidity

Values (Humidity): High, Normal

$$S = [9+, 5-]$$

$$\text{Entropy}(S) = -9/14\log_2(9/14) - 5/14\log_2(5/14) = 0.94$$

$$S_{\text{High}} \leftarrow [3+, 4-]$$

$$\text{Entropy}(S_{\text{High}}) = -3/7\log_2(3/7) - 4/7\log_2(4/7) = 0.9852$$

$$S_{\text{Normal}} \leftarrow [6+, 1-]$$

$$\text{Entropy}(S_{\text{Normal}}) = -6/7\log_2(6/7) - 1/7\log_2(1/7) = 0.5916$$

$$\text{Gain}(S, \text{Humidity}) = \text{Entropy}(S) - \sum_{v \in \{\text{High}, \text{Normal}\}} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S, \text{Humidity}) = 0.94 - \left(\frac{7}{14} \cdot 0.9852 \right) - \left(\frac{7}{14} \cdot 0.5916 \right)$$

$$\text{Gain}(S, \text{Humidity}) = 0.1516$$

Attribute: Wind

Values (Wind): Strong, Weak

S = [9+, 5-]

$$\text{Entropy}(S) = -9/14 \log_2(9/14) - 5/14 \log_2(5/14) = 0.94$$

S_{Strong} ← [3+, 3-]

$$\text{Entropy}(S_{\text{Strong}}) = 1.0$$

S_{Weak} ← [6+, 2-]

$$\text{Entropy}(S_{\text{Weak}}) = -6/8 \log_2(6/8) - 2/8 \log_2(2/8) = 0.8113$$

$$\text{Gain}(S, \text{Wind}) = \text{Entropy}(S) - \sum_{v \in \{\text{Strong}, \text{Weak}\}} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S, \text{Wind}) = 0.94 - \left(\frac{6}{14} \cdot 1.0 \right) - \left(\frac{8}{14} \cdot 0.8113 \right)$$

$$\text{Gain}(S, \text{Wind}) = 0.0478$$

Information Gain Values

$$\text{Gain}(S, \text{Outlook}) = 0.2464$$

$$\text{Gain}(S, \text{Temp}) = 0.0289$$

$$\text{Gain}(S, \text{Humidity}) = 0.1516$$

$$\text{Gain}(S, \text{Wind}) = 0.0478$$

Attribute: Temp

Values (Temp): Hot, Mild, Cool

S_{Sunny} = [2+, 3-]

$$\text{Entropy}(S_{\text{Sunny}}) = -2/5 \log_2(2/5) - 3/5 \log_2(3/5) = 0.97$$

S_{Hot} ← [0+, 2-]

$$\text{Entropy}(S_{\text{Hot}}) = 0.0$$

$S_{Mild} \leftarrow [1+, 1-]$

$Entropy(S_{Mild}) = 1.0$

$S_{Cool} \leftarrow [1+, 0-]$

$Entropy(S_{Cool}) = 0.0$

$$Gain(S_{Sunny}, Temp) = Entropy(S_{Sunny}) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S_{Sunny}|} \cdot Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp) = 0.97 - \left(\frac{2}{5} \cdot 0.0 \right) - \left(\frac{2}{5} \cdot 1.0 \right) - \left(\frac{1}{5} \cdot 0.0 \right)$$

$$Gain(S_{Sunny}, Temp) = 0.570$$

Attribute: Humidity

Values (Humidity): High, Normal

$S_{Sunny} = [2+, 3-]$

$Entropy(S_{Sunny}) = -2/5 \log_2(2/5) - 3/5 \log_2(3/5) = 0.97$

$S_{High} \leftarrow [0+, 3-]$

$Entropy(S_{High}) = 0.0$

$S_{Normal} \leftarrow [2+, 0-]$

$Entropy(S_{Normal}) = 0.0$

$$Gain(S_{Sunny}, Humidity) = Entropy(S_{Sunny}) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S_{Sunny}|} \cdot Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = 0.97 - \left(\frac{3}{5} \cdot 0.0 \right) - \left(\frac{2}{5} \cdot 0.0 \right)$$

$$Gain(S_{Sunny}, Humidity) = 0.97$$

Attribute: Wind

Values (Wind): Strong, Weak

$S_{\text{Sunny}} = [2+, 3-]$

$\text{Entropy}(S_{\text{Sunny}}) = -2/5 \log_2(2/5) - 3/5 \log_2(3/5) = 0.97$

$S_{\text{Strong}} \leftarrow [1+, 1-]$

$\text{Entropy}(S_{\text{Strong}}) = 1.0$

$S_{\text{Weak}} \leftarrow [1+, 2-]$

$\text{Entropy}(S_{\text{Weak}}) = -1/3 \log_2(1/3) - 2/3 \log_2(2/3) = 0.9183$

$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = \text{Entropy}(S_{\text{Sunny}}) - \sum_{v \in \{\text{Strong}, \text{Weak}\}} \frac{|S_v|}{|S_{\text{Sunny}}|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = 0.97 - \left(\frac{2}{5} \cdot 1.0 \right) - \left(\frac{3}{5} \cdot 0.9183 \right)$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = 0.0192$$

Information Gain Values (Sunny Subset)

Attribute: Temp

Values (Temp): Hot, Mild, Cool

$S_{\text{Rain}} = [3+, 2-]$

$\text{Entropy}(S_{\text{Rain}}) = -3/5 \log_2(3/5) - 2/5 \log_2(2/5) = 0.97$

$S_{\text{Hot}} \leftarrow [0+, 0-]$

$\text{Entropy}(S_{\text{Hot}}) = 0.0$

$S_{\text{Mild}} \leftarrow [2+, 1-]$

$\text{Entropy}(S_{\text{Mild}}) = -2/3 \log_2(2/3) - 1/3 \log_2(1/3) = 0.9183$

$S_{\text{Cool}} \leftarrow [1+, 1-]$

$\text{Entropy}(S_{\text{Cool}}) = 1.0$

$$Gain(S_{Rain}, Temp) = Entropy(S_{Rain}) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S_{Rain}|} \cdot Entropy(S_v)$$

$$Gain(S_{Rain}, Temp) = 0.97 - \left(\frac{0}{5} \cdot 0.0 \right) - \left(\frac{3}{5} \cdot 0.9183 \right) - \left(\frac{2}{5} \cdot 1.0 \right)$$

$$Gain(S_{Rain}, Temp) = 0.0192$$

Attribute: Humidity

Values (Humidity): High, Normal

$S_{Rain} = [3+, 2-]$

$Entropy(S_{Rain}) = -3/5 \log_2(3/5) - 2/5 \log_2(2/5) = 0.97$

$S_{High} \leftarrow [1+, 1-]$

$Entropy(S_{High}) = 1.0$

$S_{Normal} \leftarrow [2+, 1-]$

$Entropy(S_{Normal}) = -2/3 \log_2(2/3) - 1/3 \log_2(1/3) = 0.9183$

$$Gain(S_{Rain}, Humidity) = Entropy(S_{Rain}) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S_{Rain}|} \cdot Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = 0.97 - \left(\frac{2}{5} \cdot 1.0 \right) - \left(\frac{3}{5} \cdot 0.9183 \right)$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

Attribute: Wind

Values (Wind): Strong, Weak

$S_{Rain} = [3+, 2-]$

$Entropy(S_{Rain}) = -3/5 \log_2(3/5) - 2/5 \log_2(2/5) = 0.97$

Prepared by Madan Tiwari, MCA

$S_{\text{Strong}} \leftarrow [0+, 2-]$

$\text{Entropy}(S_{\text{Strong}}) = 0.0$

$S_{\text{Weak}} \leftarrow [3+, 0-]$

$\text{Entropy}(S_{\text{Weak}}) = 0.0$

$$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = \text{Entropy}(S_{\text{Rain}}) - \sum_{v \in \{\text{Strong}, \text{Weak}\}} \frac{|S_v|}{|S_{\text{Rain}}|} \cdot \text{Entropy}(S_v)$$

$$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = 0.97 - \left(\frac{2}{5} \cdot 0.0 \right) - \left(\frac{3}{5} \cdot 0.0 \right)$$

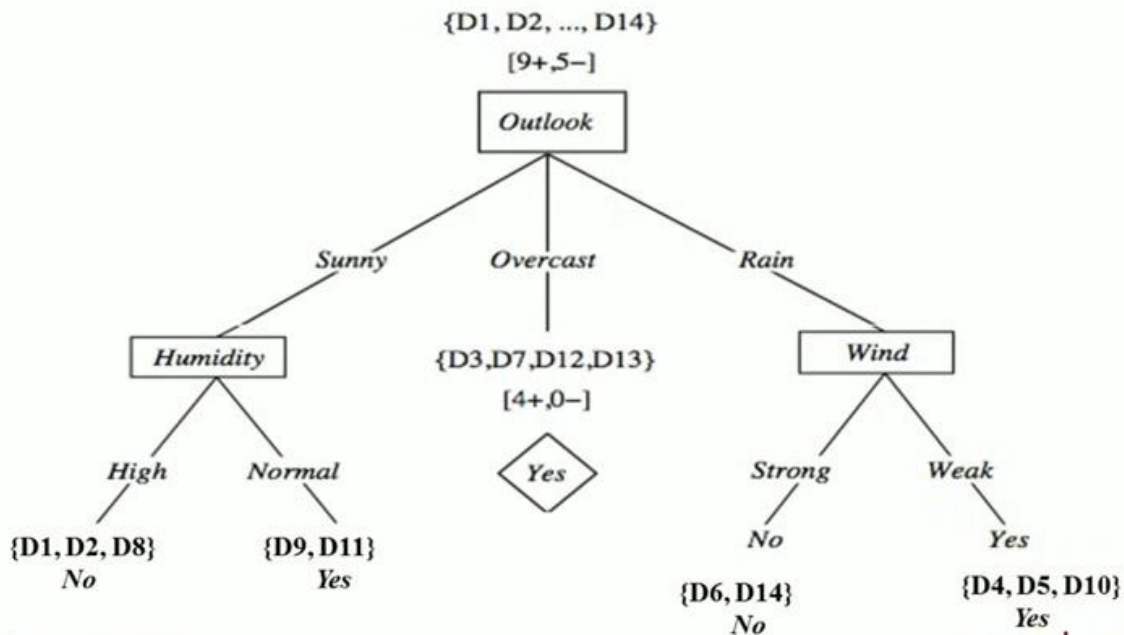
$$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = 0.97$$

Information Gain Values (Rain Subset)

$\text{Gain}(S_{\text{Rain}}, \text{Temp}) = 0.0192$

$\text{Gain}(S_{\text{Rain}}, \text{Humidity}) = 0.0192$

$\text{Gain}(S_{\text{Rain}}, \text{Wind}) = 0.97$



Hypothesis Space in Decision Trees

- The hypothesis space refers to the collection of potential decision trees that a machine learning algorithm explores during the process of constructing a decision tree.
- Specifically, we'll focus on the ID3 algorithm (Iterative Dichotomiser3), which is a classification technique used to create decision trees.

Key Features

1. Representation:

- Each decision tree corresponds to a single hypothesis about the target function.
- The hypothesis space includes all possible trees that can be formed using the attributes and their values.

2. Flexibility:

- Decision trees can represent complex functions, including disjunctions of conjunctions of attribute constraints.

3. Search Space:

- The algorithm searches through this hypothesis space to find the tree that best fits the training data.

4. Inductive Bias:

- Decision tree algorithms, like ID3 have an inductive bias favoring simpler trees and those that maximize information gain.

Challenges

- The hypothesis space is vast, especially for datasets with many attributes and values.
- Overfitting can occur if the algorithm selects overly complex trees.

Inductive bias in decision tree learning

- Inductive bias in decision tree learning represents the assumptions and preferences that guide the algorithm in generalizing from the training data.
- In the case of decision trees, this bias includes preferences for simpler trees and those that prioritize attributes with higher information gain.
- These biases help the algorithm efficiently search the hypothesis space and make predictions for unseen data.
- For decision trees specifically, this bias often includes:
 - **Preference for simpler trees:** Decision tree algorithms tend to favor smaller trees with fewer nodes and branches as these are less complex and less likely to overfit the data.
 - **Greedy attribute selection:** The algorithm often prioritizes attributes with higher information gain or purity, selecting the best attribute at each step to split the data.
 - **Hierarchical structure:** Decision trees assume that the relationships between attributes can be represented in a hierarchical manner, with decisions flowing from root to leaves.
 - These biases enable decision trees to perform well on many tasks by efficiently searching through the hypothesis space. However, the biases may also limit their ability to capture certain complex patterns in data.